

מדרִיךְ טכני — WiFiGate

WiFiGate — Technical Handbook

מדריך מקצועי מקצה לקצה — WiFiGate מחברת

איך הכול עובד, **למה** זה בנוי ככה, אילו ספריות בשימוש, כל ה-WiFiGate מסמך לימוד מלא של מערכת והלוגים החשובים לכל שלב. נכתב מתוך קריאה של קוד המקור בפועל, לא מהזיכרון. אם אתם flows, המסמך הזה הוא שכבה עמוקה יותר: מדריך README.md כללי — קראו קודם את README קוראים reference. GB An English version of this handbook is available at WIFIGATE_HANDBOOK_EN.md.

תוכן עניינים

1. ולמה WiFiGate תמונת-על: מה זו
2. הטכנולוגי — כל ספרייה ולמה בחרו בה Stack-ה
3. מפת הקוד — איפה כל דבר יושב
4. Gate, Role, Group, Dual Control — מודל הנתונים
5. ואיך זה עובד local-first שכבת האחסון — למה
6. המותאם אישית store-ה — state ניהול
7. המרכזיים מקצה לקצה Flows-ה
8. לעומק — SECURE_V3 המאובטח BLE-פרוטוקול ה
9. הצפנת קישורי הזמנה — לעומק
10. איך קישור הופך למסך — Deep Linking
11. פתיחה אוטומטית לפי קרבה — Auto Open
12. עדכון קושחה — FOTA / OTA
13. עמוד ההגדרה המקומי — Nearby Wi-Fi Setup
14. לוגים חשובים — קטלוג מלא לפי תגית
15. הרשאות ופלטפורמה
16. Build Inputs-סודות ו
17. Release-פיתוח מקומי, בדיקות ו
18. גבולות אבטחה — מה מוגן ומה לא
19. הפניות מהירות — Cheat Sheet

1. תמונת-על

לשליטה בשערים פיזיים שמונעים על ידי קושחת **React Native / Expo** היא אפליקציית WiFiGate האפליקציה היא בעצם שני דברים במקביל. (ESP32-C6 מבוססת) WiFiGate

20. על הטלפון — רשימת שערים עם רמות גישה ותוקף שונים (gate wallet) **ארנק שערים**
21. בפרוטוקול Bluetooth Low Energy **לקוח שליטה מאובטח** מול המכשיר — פותח שער דרך

מוצפן.

שלושת התפקידים של האפליקציה

ומשתמשת ברשימה הזאת כדי להניע שלוש (local-first) האפליקציה שומרת מטא-דאטה של שערים **מקומית** משימות:

- (הליבה) **BLE פתיחת שער דרך**
- **שיתוף /קבלת גישה דרך** קישורי הזמנה מוצפנים

- (<http://192.168.4.1/> עמוד) Wi-Fi מקומי דרך setup-כניסה ל

עיקרון על: הפרדת שכבות אבטחה

זו נקודה קריטית להבנת כל הארכיטקטורה — יש שתי בעיות אבטחה נפרדות עם שני פתרונות נפרדים

מנגנון	מה מגנה	שכבה
ECDH + HKDF + AES-256-GCM + HMAC	תקשורת טלפון ↔ שער	BLE (SECURE_V3)
עם מפתח נגזר AES-256-GCM URL-BLE, payload ה-URL m-HMAC URL SECRET	URL-בתוך ה payload	קישורי הזמנה

ראו סעיף. URL-לא מגן על ה SECURE_V3, BLE-הן לא מחליפות זו את זו. קישור מוצפן לא מגן על ה 18.



הטכנולוגי Stack-ה 2.

להלן הספריות המשמעותיות ולמה הן שם. package.json: מקור

ליבה — Runtime ו-Navigation

למה זה	תפקיד	ספרייה
Expo build dev-client, נותן (EAS), בלי config plugins ו- ידנית native-לגעת ב	הבסיס	<code>expo ~54 / react-native 0.81 / react 19</code>
route; הוא app-כל קובץ ב מובנה deep-linking	file-based ניתוב	<code>expo-router ~6</code>
חובה — האפליקציה תלויה native (BLE, Wi-Fi) במודולים Expo Go-שלא קיימים ב	dev build מותאם	<code>expo-dev-client</code>
UI, drag-to-reorder חלק	gestures-אנימציות ו	<code>react-native-reanimated / gesture-handler / worklets</code>
הראשי gate store הערה: ה בנוי ידנית ולא (<code>gatesStore.ts</code>) zustand-משתמש ב ראו סעיף 6	state (זמין)	<code>zustand</code>

קריפטוגרפיה — הלב של האבטחה

תפקיד	ספרייה
(ECDH ל-handshake) P-256 עקומת	@noble/curves
SHA-256, HKDF, HMAC	@noble/hashes
AES-256-GCM	@noble/ciphers
חובה (crypto.getRandomValues ל-polyfill -שיטען ראשון — ראו שורה 1 ב app/_layout.tsx)	react-native-get-random-values
וקישורים BLE ל-base64 קידוד	buffer / base-64

React-ומתאימות ל, native, audited, ללא תלות ב, JS-טהורות ב crypto כי הן ספריות @noble/*-בחרו ב
Native (מלא crypto שאין בו).

התקשורת הפיזית — BLE ו-Wi-Fi

תפקיד	ספרייה
scan, connect, GATT, notify/write BLE-ליבת ה	react-native-ble-plx
ובקשת הפעלה (on/off) Bluetooth-מצב ה	react-native-bluetooth-state-manager
scan/connect של השער AP-ל (flow ה-Nearby setup)	react-native-wifi-reborn
המקומי 192.168.4.1 setup-הצגת עמוד ה	react-native-webview

Auth-זהות ו

תפקיד	ספרייה
SMS OTP-אימות טלפון ב	@react-native-firebase/auth + firebase
(BLE-מילוי אימייל ל) Google זהות	@react-native-google-signin/google-signin
Sign in with Apple	expo-apple-authentication
OAuth flows	expo-auth-session
נירמול/ולידציה של מספרי טלפון וקידומות מדינה	libphonenumber-js + country-list

UI-קלט/פלט ו

בחירת (expo-contacts), (לשיתוף QR יצירת) react-native-qrcode-svg, (QR סריקת) expo-camera, (קבצים / CSV ייצוא) expo-file-system + expo-sharing, (מוזמן מאנשי קשר
draggable-flatlist), (לטיית) lottie-react-native, (סידור מחדש של שער) react-native-async-storage/async-storage (אחסון מקומי).

3. מפת הקוד

```
app/  
  _layout.tsx      # Expo Router — route כל מסך הוא  
  auto-open boot   # root: deep-link routing, auth gating, splash,  
  index.tsx        # מסך הבית: רשימת שערים, קבוצות, חיפוש  
  scan.tsx         # QR + import סריקת  
  gate/[id].tsx    # import פרטי שער  
  open-gate/[id].tsx # פתיחה מהירה (m-widget/deep link)  
  group/[id].tsx   # מסך קבוצה  
  dual-control/[id].tsx # dual-control מסך
```

```

nearby-wifigate-link.tsx # כניסה ל-Nearby Wi-Fi setup
wifigate-setup.tsx      # של עמוד ההגדרה המקומי WebView-ה
firmware-update.tsx     # OTA מסך
settings/[id].tsx       # הגדרות שער
auth/login.tsx          # login (provider)
auth/phone.tsx          # טלפון OTP

src/
  models.ts             # types: Gate, UserRole, GateGroup, כל ה
DualControl...
  storage.ts            # AsyncStorage – keys, load/save, כל ה
  gatesStore.ts         # state store (observable) + merge המרכזי
rules
  gateService.ts        # guest access ולידצית
  openGateFlow.ts       # orchestration-שער של פתיחת
(permissions→BT→BLE)
  gateOpenBlockReason.ts # block reasons חילופי הודעות שגויה ל
  gateSecurity.ts        # security bundle (SECURE_V3) פרטינג/נרמול של

  ble/
    openGate.ts          # BLE (scan/connect/handshake/frames) – ליבת ה
    secureProtocol.ts     # SECURE_V3 (handshake, keys, frames) הקריפטו של
    bluetoothState.ts     # Bluetooth + ensureBluetoothReady מצב
    permissions.ts        # BLE הרשאות

  inviteLinkCrypto.ts     # (AES-GCM) קישורים payload הצפונה/פענוח של
  linkService.ts          # פרטינג/בנייה של קישורי הזמנה
(single/group/dual)
  dualControlInviteImport.ts # import של הזמנת dual-control
  groupInviteImport.ts     # import של הזמנת קבוצה

  auth/                  # LocalAuthContext, providerIdentity,
google/apple
  auto-open/              # runtime + native bridges + headless task +
config
  fota/                   # publicFota (Firebase), driveFota (legacy)
  widgets/               # widget שער (Android) + headless open
  wifi/                  # useGateWifiConnectionGuard
  crypto/                # hmacSecrets (שחזור) HMAC_URL_SECRET
  i18n/                  # שפות ותרגומים
  permissions/           # appLaunch, contacts, wifi
  utils/                 # datetime, gateCommand, guestAccess,
subscription...
  components/            # UI: modals, gate-settings, admin-access... כל ה
  features/my-gates/     # collections-הבית של מסך (tiles, helpers)

docs/
  ble-security-protocol-v3.md # SECURE_V3 המפרט הפורמלי של
  WIFIGATE_HANDBOOK_HE.md    # המסמך הזה

README.md                # סקירה כללית
README_WIFIGATE_LINK_INVITATIONS.md # פרטוקול קישורי ההזמנה במלואו
README_OTA_FIREBASE.md   # OTA n-Firebase Storage
WIFIGATE_API_README.md   # WIFIGATE API קישורים שהונפקו ע"י

```

4. מודל הנתונים

מקור: src/models.ts.

4.1 Gate

:הישות המרכזית. שדות חשובים

```

type Gate = {
  id: string; // כרגע מזהה מקומי (= systemId)
  systemId: string; // למשל BLE-מזהה החומרה, משמש ב
  "9876543210123456"
  systemName: string; // השם המקורי/מיובא
  displayName: string; // השם שהמשתמש יכול לערוך
  role: UserRole; // ★ 4.2 ראו
  subscription?: "OFFLINE" | "ONLINE" | "WIFI" | "ALL_IN_ONE";
  security?: GateSecurityConfig; // SECURE_V3 של bundle-ה
  gateCommand?: GateCommandType; // סוג הפעולה שהקושחה דיווחה
  (PULSE/TOGGLE/PUSH_BUTTON...)
  autoOpen?: GateAutoOpenSettings;
  // שיתוף/אירוח:
  hostPhone/hostName/hostEmail? // המזמין
  guestPhone? // פרטי guest בקישור) המוזמן
  accessStart/accessEnd? // (ms) חלון תוקף
  // PRE_USER/PRE_ADMIN-רישום ל
  residentName/floorNumber/apartmentNumber/parkingNumber?
  openHistory?: GateOpenHistoryEntry[]; // פתיחות אחרונות
  (newest-first)
};

```

4.2 Role — התפקיד קובע הכול

```
type UserRole = "ADMIN" | "USER" | "GUEST" | "PRE_USER" | "PRE_ADMIN";
```

תפקיד	משמעות	מקור	אחרי BLE פקודת import
ADMIN	מנהל מאושר	אחרי אישור קושחה	OPEN / TOGGLE * / AUTO_OPEN
USER	משתמש קבוע מאושר	אחרי אישור קושחה	OPEN / TOGGLE * / AUTO_OPEN
GUEST	גישה זמנית עם חלון זמן	guest קישור	OPEN_GUEST / TOGGLE * GUEST
PRE_USER	טרם, user-הוזמן כ אושר	user קישור (r=PU)	ADD_USER
PRE_ADMIN	admin-הוזמן/נוסף כ טרם אושר	או קישור QR/manual admin (r=PA)	ADD_ADMIN

היא לא יכולה לדעת שהקושחה אישרה אותך. עד שהמכשיר — local-first כי האפליקציה `PRE_*` למה `**` בהצלחה מול השער, הקושחה מאשרת, וה `ADD_USER/ADD_ADMIN` מאשר, אתה במצב "ממתין". כשמריצים `maybeUpgradeRole` (ראו `PRE_USER→USER / PRE_ADMIN→ADMIN` משדרג `onRoleUpgrade` callback ב- `openGateFlow.ts` ב- `openGate.ts` ואת `persist` ב- `openGateFlow.ts`).

4.3 חוקי מיזוג תפקידים (למה שער לא "נהרס")

בטעות `downgrade` המערכת מגנה על גישה קבועה מפני `addGate` ב- `gatesStore.ts`: מקור:

- `guest` כשמייבאים קישור `GUEST`-לא יורד ל `(ADMIN/USER)` שער קבוע
- חלש יותר `PRE_*` תפקיד קבוע לא נדרס ע"י תפקיד
- `pending`/כן יכול להשתדרג לתפקיד קבוע `guest` שער
- מאוחר מביא רק שם ברירת-מחדל `import` נשמר אם `(displayName)` שם מותאם אישית

4.4 GateGroup i-DualControl

- ניתן לקינון BLE/מקומי בלבד. לא משנה שום התנהגות קושחה UI קונטיינר — `GateGroup`

`parentGroupId` דרך `gateIds[]` `i-dualControlIds[]` מכיל.

- עם שתי תוויות פעולה (`firstGateId/secondGateId`) זוג שערים — **DualControl**

מכילה בדיוק 2 שערים dual-control כל הזמנת. ("למשל "פתח"/"סגור", `firstLabel/secondLabel`)

5. שכבת האחסון

לא בענן. אין "בעלות", AsyncStorage-מצב השערים חי ב — **local-first** המערכת. מקור: `src/storage.ts`.
שער "מרכזית בשרת".

(KEYS) מפתחות האחסון

```
wifigate:gates # מערך השערים
wifigate:localProfile # פרופיל onboarding
(name/email/phone/provider/terms)
wifigate:phone # מספר טלפון
wifigate:username # שם תצוגה
wifigate:language / :languagePreview
wifigate:gates:order # סדר תצוגה
wifigate:gates:groups # קבוצות
wifigate:gates:dualControls # dual controls
wifigate:gates:toggleRelayStates # TOGGLE אחרון לשערי relay מצב
wifigate:firmware:url # קושחה אחרון URL
wifigate:defaultCountryIso2 # קידומת מדינה ברירת מחדל
wifigate:appleIdentityRecords # Apple (Apple זהות-userId מיפוי לא שולח אימייל)
(בכל פעם)
wifigate:guestTable:* # guests העדפות תצוגת טבלת
```

נקודות עדינות שכדאי להכיר

- מנרמל, `lastOpenedAt` עם `openHistory` מנרמל בכל טעינה: `**loadGates()` ממזג `gateCommand`, `fotaChannel`, `hostEmail` (lowercase), guest access.
- כתיבה כושלת לא מפילה את — fallback בולעים שגיאות `**getJSON/setJSON` ומחזירים `debug`-אבל גם לא מתריעה — שווה לזכור ב) האפליקציה.
- **Android: מגבלת** `DEFAULT_ANDROID_ASYNC_STORAGE_LIMIT_BYTES = 6MB`.
- מצוין debug מלא — כלי `dump` + מדפיס שימוש/נותר `debugPrintAllLocalStorage()`.
- `cachedLocalProfile` אחת פעם נטען; `loadPhone/loadUsername/loadEmail` נטען פעם אחת מטמון פרופיל מטמון.
- מעדיפים אותו.

6. state ניהול

מקור: `src/gatesStore.ts`.

זהו `observable store` לא `Redux` בוני ידנית — `zustand` ולא `Redux` בוני ידנית — `observable store` זהו

```
let _gates: Gate[] = [];
const listeners = new Set<() => void>();
export function subscribe(fn) { listeners.add(fn); return () =>
listeners.delete(fn); }
function notify() { listeners.forEach(f => f()); }
```

בונה מערך חדש, (קוראת **immutably** עובדת (...), `addGate`, `updateGateRole`, `renameGate`, ...) כל מוטציה `saveGates`, `notify`, ובמידת הצורך, `syncGateWidgets()` + `syncAutoOpen()`.

מה קורה באתחול (חשוב מאוד) — `initGates()`

- 22. `loadGates()` מ-AsyncStorage.
 - 23. `normalizeGateRoleOnLoad` — ישנים `guests` מיגרציה של (`isGuest:true` → `role:"GUEST"`).
 - 24. (ברירת מחדל) `comments` מיגרציית.
 - 25. בלבד role-based המודל היום — `role**` הסרת שערים ישנים ללא**.
 - 26. `notify()`.
 - 27. Alert. **שפג תוקפם** ומראה **guest מסיר שערי** — `cleanupExpired(true)`.
 - 28. `syncGateWidgets()` + `syncAutoOpen()`.
- לוגים לחפש: `[Store] initGates()`, `[Store] N gates loaded`, `[Store] Removed N legacy gate(s)`, `[Store] N guest gate(s) removed (expired)`.

7. המרכזיים Flows-ה

7.1 Onboarding / Sign in

מסלולים: `app/auth/login.tsx`, `app/auth/phone.tsx`.

הזרימה:

- 29. נשמר אימייל מנורמל בפרופיל → `provider` (Google/Apple) המשתמש נכנס דרך
 - 30. שם פרופיל → קידומת מדינה → מספר טלפון → אישור תנאי שימוש
 - 31. אישור קוד → SMS OTP (Firebase Auth)
 - 32. `saveLocalOnboardingProfile` ל `Auto Open native`-שומר את הכול, מסנכרן טלפון/שם ל
- `host phone + host email` (מאובטחת, הקושחה מאמתת את זוג המזמין BLE **למה טלפון+אימייל?** בפתיחת `guests` עבור `email` `command`-לכן האימייל נשמר ומועבר ב).

אם `/auth/phone` -מפנה ל `onboarded` → אם לא: `LayoutNavigator` (`_layout.tsx` -עצמו ב `gating`-ה או `/auth/login` או `provider identity` כבר יש).

7.2 הוספת שער (QR / Manual)

- `Manual/QR` → `addGateFromQrOrManual` → `role PRE_ADMIN`.
- -מנורמל ל (`protocol/keyId` בלי, `dpk/psk`) קומפקטי `security bundle` שמכיל QR
- `gateSecurity.ts` (ראו `keyId="key-1"` עם `SECURE_V3`)
- `SECURE_V3` שערים חדשים חייבים — `security bundle` דוחה שער ללא `import/save`

7.3 Import של קישור הזמנה (single/group/dual)

מקורות: `linkService.ts`, `inviteLinkCrypto.ts`, `_layout.tsx`, `[app/gate/[id].tsx](../app/gate/[id].tsx)`.
`README_WIFIGATE_LINK_INVITATIONS.md` -פרוטוקול מלא ב

הזרימה:

33. `wifigate://` או `(-api/)` או `wifigate.io/wifigate-link/` פותח OS-ה
34. `(single/group/dual/open-gate)` מזהה את הסוג `extractInvitationRoute` → `_layout.tsx`
35. `onboarding` עד סיום `(pendingInviteUrl)` הקישור **נכנס לתור** → `onboarded` אם לא
36. `(group/dual)` או `app/gate/[id].tsx` -ניתוב ל
37. קומפקטי JSON-ל `(AES-GCM)` `ep=` מפענח `parseIncomingLink`
38. `(private vs public)` **ולידציה**: פענוח תקין? חלון זמן? הרשאת טלפון
39. `(PRE_ADMIN)` — `addGateFromGuestInvite` / `addGateFromUserInvite` / `addGate`

לפי חוקי המיזוג state-ממזג לתוך ה

Public vs Private (מקור: פרוטוקול הקישורים):

- יכול לקבל `(-p)` טלפון יעד (קיים) → הזמנה **פרטית**: רק הטלפון המטמון שתואם ל `p` אם שדה
- `(-g)` חסר → **ציבורית**: מוגבלת רק בחלון הזמן `p` אם

admin בפועל: הזמנת. `code role` `A→ADMIN`, `U→USER`, `G→GUEST`, `PU→PRE_USER`, `PA→PRE_ADMIN`.
 כ-`g` guest, כ-`r="PU"` user, כ-`r="PA"` נשלחת כ

7.4 המלא flow-ה — BLE פתיחת שער דרך

`Orchestration` ב-`openGateFlow.ts`. החשוב ביותר flow-זהו ה

```
runGateOpenFlow(gate)
├─ ensureBlePermissionsInteractive() → "permissions_blocked"
├─ ensureBluetoothReady() → "bluetooth_off"
├─ loadPhone() → "phone_missing"
├─ getGuestAccessValidation(gate) → "guest_expired"
├─ טוען email/username/toggleRelayState
├─ openGateBLE({...}) → ble/openGate.ts
│   └─ (onRoleUpgrade → updateGateRole)
│   └─ toggleRelayState אם gateCommand=TOGGLE
│   └─ אם OK ולא PRE_* → markGateOpened (openHistory)
```

ואם ההודעה (`OK→ok`, `UNAUTHORIZED→unauthorized`, `TIMEOUT→timeout`, `FAIL→fail`) מיפוי סטטוסים
`event_active/sensor_inactive` → מעידה על אירוע פעיל/חיישן

בתוך `openGateBLE` (ב-`ble/openGate.ts`):

Scan / Connect — `findConnectedGate`: מתאים לפי `systemId` devices, סורק 40.

אם יש `(slow-android)` ואיטי `(default)` מהיר `profile` מתחבר. יש `(advertise-שם ה)`
`preferredDeviceId` (מ-Auto Open) — מנסה חיבור ישיר קודם

Discover GATT — `Nordic UART`-מגלה את שירות ה, `(Android)` `MTU 517` מבקש 41

`Service 6e400001-...` 42

`RX 6e400002-...` (כתיבה מהאפליקציה לשער) 43

`TX 6e400003-...` (מהשער לאפליקציה `notify`) 44

Inbox — `createBleTextInbox`: מ `chunks`-מרכיב הודעות מ: `\n` לפי 45

מקומי `echo` ומסנן, `(fallback)` `polling` וגם ב `notify`-ב

Handshake SECURE_V3 — `client_hello` → `device_hello` → `session keys` (8 סעיף) 46

Encrypted command frame — `authTag` בונה את הפקודה, מוסיף `(seq=1)` מצפין ושולח 47

Encrypted response frame — `SecureDeviceResponse` קורא, מפענח, מחזיר 48

בחירת הפקודה לפי תפקיד וסוג שער 7.5

מציב	BLE פקודת
שער רגיל, USER/ADMIN	OPEN (או AUTO_OPEN)
TOGGLE שער, USER/ADMIN	TOGGLE_OPEN / TOGGLE_CLOSE (לפי השמור toggleRelayState)
PUSH_BUTTON שער, USER/ADMIN	PUSH_BUTTON_HOLD (חוזר) + PUSH_BUTTON_RELEASE
שער רגיל, GUEST	OPEN_GUEST
TOGGLE שער, GUEST	TOGGLE_OPEN_GUEST / TOGGLE_CLOSE_GUEST
PRE_USER	ADD_USER (→ מאשר)
PRE_ADMIN	ADD_ADMIN (→ מאשר)

והקושחה — `guestName`, `inviterName`, `inviterPhone`, `inviterEmail` מעביר בנוסף `Guest open` האפליקציה עוצרת עם `guest` — בהזמנת `inviterEmail` מאמתת את זוג המזמין לפני פתיחה. אם חסר הודעה לבקש הזמנה חדשה.

HOLD של frame כל: (`docs/ble-security-protocol-v3.md`: מקור) הוא מקרה מיוחד `Push button` frame; עולה בכל `seq`-נשאר פתוח כל עוד לוחצים; ה BLE אותו חיבור; TTL-session-מרענן את ה `finger-up` של 1 watchdog לקושחה יש `PUSH_BUTTON_RELEASE`. נשלח

שיתוף גישה (יצירת קישורים) 7.6

ב- `buildShareData` (single), `buildGroupShareData`, `buildDualControlShareData` מקורות: `linkService.ts`.

Nearby Wi-Fi setup וקישור, admin קישור, user קישור, guest יכול ליצור: קישור Admin ל- `encryptInviteLinkPayload` -מוצפן ב, (`i,n,r,h,e,p,kid,dpk,psk...`) נבנה קומפקטי URL-ומשולב ל, `ep=v1.<base64url>`, הציבורי.

המאובטח BLE-פרוטוקול ה. 8

מקורות: `src/ble/secureProtocol.ts` + `docs/ble-security-protocol-v3.md` המפרט.

v3. הוא הפרוטוקול המאובטח היחיד. גם האפליקציה וגם הקושחה חייבות `SECURE_V3`.

8.1 Provisioning bundle

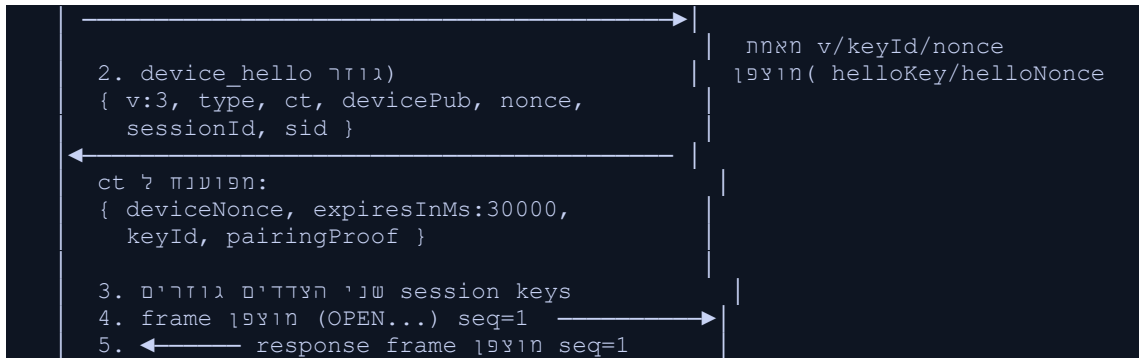
`Gate.security`-כל שער מאובטח מחזיק ב:

```
{ protocol: "SECURE_V3",
  keyId: string,           // חריץ מפתח, למשל "key-1"
  devicePublicKey: string, // דחוט P-256 מפתח ציבורי, hex
  pairingSecret: string }  // בייט לכל שער 32 pairing סוד, hex
```

8.2 Handshake — צעד אחר צעד

`wifigate-ble-secure-v3`: תוויות, msברירת מחדל 15000 freshness, גרסה 3: (`secureProtocol.ts`) קבועים `secure-v3` (transcript), `device-hello-proof-v3`, `command-auth-v3`.

שער	אפליקציה (ESP32)
1. client_hello (plaintext JSON)	
{ v:3, type, keyId, sessionId, clientPub, clientNonce }	



8.3 גזירת המפתחות (key schedule)

```

ephemeral P-256 keypair (client)
sharedSecret = ECDH(clientSecret, devicePub).X // של הנקודה (X) בייט 32
(הדחוסה)

helloKey = HKDF-SHA256(sharedSecret, salt=clientNonce, info="hello-key|sid|session|keyId", 32)
helloNonce = HKDF-SHA256(sharedSecret, salt=clientNonce, info="hello-nonce|...", 12)

// device_hello מופיע:
plaintext = AES-256-GCM(helloKey, helloNonce,
AAD="device_hello|systemId|sessionId").decrypt(ct)

// pairingSecret (proof-of-possession) אימות זהות המכשיר:
pairingProof = HMAC-SHA256(pairingSecret, "device-hello-proof-v3|systemId|session|keyId|clientPub|clientNonce|devicePub|deviceNonce")

transcriptHash = SHA256("wifigate-ble-secure-v3|systemId|session|keyId|clientPub|clientNonce|devicePub|deviceNonce")

// session מפתחות (salt = clientNonce || deviceNonce):
clientWriteKey = HKDF(sharedSecret, salt, "client-write-key|...", 32)
deviceWriteKey = HKDF(sharedSecret, salt, "device-write-key|...", 32)
clientNoncePrefix = HKDF(sharedSecret, salt, "client-nonce-prefix|...", 8)
deviceNoncePrefix = HKDF(sharedSecret, salt, "device-nonce-prefix|...", 8)
authKey = HKDF(pairingSecret, salt=transcriptHash, "command-auth-key|...", 32)
  
```

8.4 מסגרת פקודה מוצפנת (command frame)

```

command = { cmd, systemId, timestamp, freshnessMs:15000, phone, role, name, ... }
commandHash = SHA256(stableJson(command)) // יציב JSON
(מפתחות מחוייבים)
authTag = HMAC-SHA256(authKey, "command-auth-v3" || transcriptHash || seq(4B BE) || commandHash)

innerPayload = { authTag, command }
nonce = clientNoncePrefix(8) || seq(4B BE) // בייט 12
ct = AES-256-GCM(clientWriteKey, nonce,
AAD="frame|c2d|systemId|sessionId|seq").encrypt(innerPayload)
frame = { v:3, type:"frame", seq, ct }
  
```

{ הקושחה מחזירה. d2c עם AAD, deviceWriteKey, deviceNoncePrefix, סימטריה (d2c) התגובה }
 type:"response", cmd, status:OK|FAIL|UNAUTHORIZED, gateCommand?, relayState?, ... }.

חשוב: הפקודה מסודרת עם מפתחות ממויינים אלפביתית וכלי **stableJson** (HMAC-יהיה זהה בשני הצדדים. שינוי סדר שדות ישבור את ה-SHA256-כדי ש, **stableStringify**)
 session-כשאפשר לגזור אותו מה wire-על ה command-מושגת מה systemId, בנוסף.

8.5 Freshness ו-Replay-עמידות ל

Replay resistance נובעת משכבות מרובות: `sessionId` טרי, `clientNonce/deviceNonce` טריים, `seq` freshness ואכיפת, session TTL, transcript-קשור ל `authTag`, frame-פר.

Freshness או session-של ה-TTL חסר/0, אם הוא גדול מ `freshnessMs` (הקושחה: דוחה פקודה אם **Freshness** המבוקש, את הגיל הנמדד, `freshnessMs` הקושחה מתעדת את `freshnessMs`-גדול מ session-אם גיל ה ואת סיבת הדחייה.

8.6 (device key rotation) שחזור מפתח מכשיר

תקין — האפליקציה מקבלת את `pairingProof`-שונה מהשמור אבל ה `devicePub` אם השער מחזיר ומרעננת אותו לאחר הצלחת הפקודה. אם הפענוח (`usedStoredDevicePublicKey=false`) המפתח החדש → שגיאה ידיונית "This gate's secure identity changed. Scan the latest WIFIGATE QR..." (reprovision/flash-format המכשיר כנראה עבר).

8.7 (BLE transport) פרמטרים של התעבורה

להודעה `openGate.ts`: MTU 517 (מבוקש ל-512 attribute), `chunk = mtu-3`, `timeout = 10000ms` (לפי המאפיין. הודעות מיוחדות response כתיבה עם/בלי, `\n` פריימינג לפי, `device_hello`, `[BLE] [SECURE]` הודעות לא צפויות מדולגות עם — `systemId/sessionId/seq` מסוננות לפי `frame` Ignoring unexpected...

9. הצפנת קישורי הזמנה

מקור: `src/inviteLinkCrypto.ts`.

```
key = HKDF-SHA256(HMAC_URL_SECRET_bytes,
  salt="wifigate-invite-link:key-salt",
  info="wifigate-invite-link:aes-256-gcm", 32)
ciphertext = AES-256-GCM(key, managedNonce, AAD="wifigate-invite-link:v1").encrypt(JSON)
token = "v1." + base64url(nonce||ciphertext)
```

- אוטומטית ciphertext-מוגרל ומוצמד לפני ה nonce — `managedNonce (gcm)`
- עתידי version bump מאפשר — `v1` הפלט מגורסה
- לאפליקציה `ep`-אתר האינטרנט לא מפענח — הוא רק מעביר את ה

מאפייני אבטחה ומגבלה חשובה

שיבוש גורם לכשל פענוח באפליקציה. ☒ האתר ☒ `preview`/דפדפן/`chat`-אטום ב `payload` ה- ☒ `plaintext` הסטטי לא צריך לדעת.

⚠️ אמין build לפי נמען. כל end-to-end לא — **build-שמוטמע ב shared secret מגבלה**: זו הצפנה עם רגילים, אבל לא שקול ל `query params` יכול לפענח קישורים תקינים. זה חזק מ `URL secret`-שמכיל את ה `token` חד-פעמי/מונפק-שרת/קשור-נמען.

10. Deep Linking

מקור: `extractInvitationRoute` ב `app/_layout.tsx`.

סכימות נתמכות: `wifigate://` (native) ו-`https://wifigate.io` (paths: `wifigate-link`, `wifigate-api`).

ומחליטה על היעד לפי סדר עדיפויות URL הפונקציה מפרקת: `dual-control` → `group` → `open-gate` (widget) → `gate` (כולל `?ep=` - שהופך ל `gateId="invite"`). מקובן `?link=` תומכת גם ב.

ומעובד רק אחרי סיום ה `(pendingInviteUrl)` הקישור נכנס לתור, `onboarded` עדינות: אם המשתמש לא `onboarding` (`useEffect` על `isOnboarded`). `lastHandledInviteUrlRef` כפילות מסוננת דרך.

לוגים לחפש: `[Linking] event/initial URL received`, `[Linking] Routing invitation URL`, `[Linking] Queueing invitation URL until onboarding completes`, `[Linking] Ignoring unsupported URL`.

11. Auto Open

מקורות: `src/auto-open/runtime.ts`, `config.ts`, `nativeAutoOpen.ts`, `autoOpenHeadlessTask.ts`.

האפליקציה פותחת אותו אוטומטית, (חזק מספיק RSSI) הרעיון: כשאתה מתקרב לשער.

בלבד fallback הוא בעצם JS-המשותף ב `runtime`-ארכיטקטורה חשובה: ה:

- מכובה JS runtime וה-`ensureNativeIosAutoOpenRunning` native → אם יש מודול iOS-ב
- JS וה-`ensureNativeAutoOpenServiceRunning` (foreground service) → **Android**-ב

מכובה runtime.

- סורק מועמדים — native רץ רק אם אין `(runRuntimeTick)` JS runtime-ה

`processAutoOpenTrigger`-בוחר את החזק ביותר, ומדיח ל `(scanAutoOpenCandidatesBLE)`.

לא רץ אמין ברקע, בטח לא ל React Native ב-JS — צריך לעבוד ברקע Auto Open כי **native** למה BLE scanning מתמשך.

`startAutoOpenRuntime()` מ-`_layout.tsx` (useEffect)-נקרא פעם אחת.

לוגים: `[AutoOpen] [Runtime]`, `[AutoOpen] [iOS]`, `[AutoOpen] [Android]`, `[BLE] [AUTO_OPEN]` `Candidate scan matched N gate(s)`.

12. FOTA / OTA

מקורות: `src/fota/publicFota.ts` (Firebase, החדש), `driveFota.ts` (Google Drive, legacy), `app/firmware-update.tsx`. מדריך: `README_OTA_FIREBASE.md`.

לפי שני מימדים **Firestore Storage**-הקושחה נקראת מ:

- QR ערוץ: `fotaChannel = dev / prod`.
- `variant 8`: `MB / 16MB` חומרה.

⇒ 4 feeds: `{dev|prod}-fota/{8mb|16mb}/manifest.json`. מורידה `chunks` האפליקציה מורידה. `manifest`-שרשומים ב `wgfota`. לפני המשך `flashVariantMb` ומאמתת, `manifest`-שרשומים ב `wgfota`.

→ AP-של ה SSID/password/token השער מחזיר → BLE דרך `OTA_OPEN`: הזרימה מול השער
app.json-הציבורי ב config-האפליקציה מתחברת ומזרימה את הקושחה. ה.
(`expo.extra.fotaStorage*`).

לוגים: `[FOTA]`.

13. Nearby Wi-Fi Setup

מקורות: `src/components/gate-settings/admin-access/NearbyWIFIGATELink.tsx`, `app/wifigate-setup.tsx`.

המקומי של השער ולהיכנס לעמוד ההגדרה AP-לפתוח את ה admin-מאפשר ל

- 49. "Nearby WIFIGATE Link" בוחר admin
- 50. Bluetooth ודא + BLE בקשת הרשאות
- 51. BLE דרך `WIFI_OPEN` שליחת
- 52. נוכחיים SSID/password אם השער מאשר → מחזיר
- 53. (`react-native-wifi-reborn`) סמוכות Wi-Fi רשתות scan
- 54. של השער AP-חיבור ל
- 55. WebView-ב `http://192.168.4.1/` פתיחת

app-scheme, המקומי של השער, חוסם ניווט חיצוני שרירותי, חוסם host-מוקשח: מוגבל ל **WebView**-ה, reload/recovery ומציע, CSV, תומך בהורדת

נפרדת על רשת מקומית workflow הזה — זו HTTP-לא מאבטח את עמוד ה BLE-פרוטוקול ה ⚠

לוגים: `[NearbyWIFIGATELink]`, `[WebView]`, `[WiFi]`.

14. לוגים חשובים

(BLE, אבל שים לב: הרבה מהלוגים הקריטיים. `__DEV__`-שרצים `**` רק ב `log/warn` מגדיר `logger.ts`, רגילים. לכן builds-ישירות, כלומר יופיעו גם ב `console.log/console.warn` משתמשים ב (Store, Linking) המרכזי בשטח debug-הוא כלי ה BLE-trace

14.1 קטלוג תגיות (לפי תדירות)

מה מספרת	היכן	תגית
scan, connect, MTU, chunks TX/RX, timeouts	ble/openGate.ts	<code>[BLE]</code>
handshake, client/device hello, frames מוצפנים	ble/openGate.ts	<code>[BLE]</code> <code>[SECURE]</code>
Auto Open מועמדים ל scan	ble/openGate.ts	<code>[BLE]</code> <code>[AUTO_OPEN]</code>
merge, downgrade טעינה, prevention, cleanup	gatesStore.ts	<code>[Store]</code>
של orchestration-שלבי ה הפתיחה	openGateFlow.ts	<code>[GateOpenFlow]</code>
(collections, tiles) מסך הבית	features/my-gates	<code>[MyGates]</code>
מסך פרטי שער	app/gate	<code>[GateScreen]</code>
runtime, iOS, Android, UI	auto-open/*	<code>[AutoOpen]*</code>

[Linking]	layout.tsx	וניתוב deep links קליטת
[ENCRYPTED_INVITE_PARAM]	linkService	פרסינג ep=
[InviteLinkCrypto]	inviteLinkCrypto.ts	payload כשל פענוח
[FOTA]	fota/*	OTA
[NearbyWIFIGATELink] [WebView] [WiFi]	setup flow	Nearby Wi-Fi
[GateWidget] [WidgetHeadlessOpen] [WidgetOpenGate]	widgets/*	פתיחה widget
[storage]	storage.ts	כשלי כתיבה, usage, dump
[Contacts] [Camera]	permissions	הרשאות

14.2 flow (debug playbook) מה לחפש בכל

פתיחת שער נכשלת:

```
[GateOpenFlow] Checking BLE permissions / Bluetooth state
[BLE] Scanning for gate for Nms...
[BLE] Gate not found. Move closer... ← TIMEOUT (שם/קרבה advertise)
[BLE] Requested MTU 517, negotiated N
[BLE][SECURE] Sending client_hello. sessionId=...
[BLE][SECURE] Waiting for device_hello...
[BLE][SECURE][DEVICE_HELLO_PARSE] ... ← bundle לא תואם (reprovision?)
[BLE][SECURE] Sending encrypted OPEN frame. seq=1
[GateOpenFlow] BLE result: { status: ... }
```

זהות שער השתנתה: חפש `Could not decrypt device_hello with the stored secure bundle / pairing proof mismatch` → מחדש QR צריך לסרוק.

של קישור נכשל Import: `[Linking] Routing invitation URL → [InviteLinkCrypto] Failed to decrypt invitation payload` (secret/שיבוש) או Alert "Invalid invitation / Access expired / Unauthorized".

נעלם guest: `[Store] N guest gate(s) removed (expired)` — פג תוקף בעת אתחול.

נמנע downgrade: `[Store] Preventing role downgrade for gate X: ADMIN stays.`

15. הרשאות ופלטפורמה

האפליקציה צריכה native. וקבצים app.json-מוגדר ב

- QR מצלמה — סריקת
- Bluetooth — תקשורת שער
- BLE/Wi-Fi scanning-נדרש היסטורית ל — Location (Android)
- Nearby Wi-Fi devices (Android).
- גישה ל-192.168.4.1 — Local network (iOS)

Deep linking ל מוגדר `wifigate://` ו-`https://wifigate.io`.

- שער כניסה להרשאות: `src/permissions/appLaunch.ts`

(`checkAppLaunchPermissions` = BLE + Bluetooth ready;
`prepareStartupPermissionsForSignIn` = contacts + BLE).

- Bluetooth מצב: `ble/bluetoothState.ts` (`ensureBluetoothReady`).
- BLE הרשאות: `ble/permissions.ts` (`ensureBlePermissionsInteractive`, `hasBlePermissions`).

16. Build Inputs-סודות ו

(`gitignored` חלקים) קלטים נדרשים:

- הצפנת `HMAC_URL_SECRET` סוד. משמש לשחזור `src/crypto/HMAC_SECRETS.json` —
`src/crypto/hmacSecrets.ts`. נגזר דרך `URL/HMAC legacy` קישורים (וכל הגנת
 - `GoogleService-Info.plist` (iOS) + `google-services.json` (Android) — Firebase.
- EAS:** `scripts/eas-pre-install.js` שם את `HMAC_SECRETS.json` בסביבת ה-`build` (`eas-build-pre-install`). `repo`-מחוץ ל, `CI`/מסופק דרך משתנה סביבה, `repo`-מחוץ ל.
- ⚠ זורק שגיאה וכל הצפנת/פענוח הקישורים `deriveInviteLinkKey` — לא מוגדר `HMAC_URL_SECRET` אם תיכשל.

17. פיתוח מקומי וריליז

`m-package.json`:

```
npm install
# src/crypto/HMAC_SECRETS.json (או HMAC_SECRETS_JSON ב-EAS) טפק
# לטביבה Firebase ודא קבצי

npm run start:dev-client # expo start --dev-client
npm run start:usb        # Android דרך USB (PowerShell script)
npm run reset:app        # של אנדריד אפליקציה state איפוס
npm run reset:usb        # reset + relaunch

npm run lint             # eslint
npm run typecheck        # tsc --noEmit
npm run verify           # lint + typecheck ★ לפני shipping
```

Release checklist (על מכשיר smoke test)

- ☐ Sign-in ב-phone OTP
- ☐ QR / manual import
- ☐ import של קישור מוצפן
- ☐ guest invite flow
- ☐ user/admin invite flow
- ☐ BLE-פתיחת שער ב
- ☐ Nearby Wi-Fi setup
- ☐ (אם רלוונטי) setup-מעמוד ה CSV ייצוא

smoke test. אחרי EAS עם `submit`-ו `build`.

18. גבולות אבטחה

חשוב להבין מה לא מוגן

- **HTTP setup-בלבד** — לא על עמוד ה BLE-מגן על תעבורת ה `SECURE_v3` בטלפון `credentials`-לא על שיתוף הקישורים, ולא על אחסון ה, (`192.168.4.1`)
- **build ב-shared secret**-קישורי הזמנה מוצפנים ועמידים לשיבוש, אבל משתמשים במונפק-שרת/חד-פעמי/קשור-נמען `token` לא
- אין בעלות שער מסונכרנת בענן — `AsyncStorage` ב **local-first** מצב השערים
- תפקידים נאכפים גם ע"י לוגיקת האפליקציה וגם ע"י התנהגות המכשיר, אבל ****ההרשאה**
- **UI**-הסופית של החומרה חייבת להיאכף בקושחה ****** — לא לסמוך על ה

19. Cheat Sheet

קבצים לפי משימה

קרא	...רוצה להבין
<code>docs/ble-security-protocol-v3.md</code> , <code>secureProtocol.ts</code>	<code>message flow</code> ו- <code>BLE</code> קריפטו
<code>ble/openGate.ts</code>	<code>BLE (scan/connect/frames)</code> ליבת
<code>openGateFlow.ts</code>	של פתיחה <code>orchestration</code>
<code>models.ts</code> , <code>gatesStore.ts</code>	<code>merge</code> ו- <code>role</code> חוקי
<code>linkService.ts</code> , <code>inviteLinkCrypto.ts</code>	קישורים <code>import</code> /יצירת
<code>README_WIFIGATE_LINK_INVITATIONS.md</code>	פרוטוקול קישורים מלא
<code>app/_layout.tsx</code>	<code>deep linking</code>
<code>NearbyWIFIGATELink.tsx</code> , <code>app/wifigate-setup.tsx</code>	<code>Nearby setup</code>
<code>README_OTA_FIREBASE.md</code> , <code>fota/publicFota.ts</code>	<code>OTA</code>
<code>storage.ts</code>	אחסון

BLE (SecureCommandName) פקודות

```
OPEN TOGGLE_OPEN TOGGLE_CLOSE AUTO_OPEN
OPEN_GUEST TOGGLE_OPEN_GUEST TOGGLE_CLOSE_GUEST
PUSH_BUTTON_HOLD PUSH_BUTTON_RELEASE
ADD_USER ADD_ADMIN
WIFI_OPEN OTA_OPEN GET_FW_VERSION
GET_AUTO_OPEN_STATUS CHECK_AUTO_OPEN_RSSI GET_GATE_COMMAND
```

gateCommand ערכי בתגובה

1=PULSE 2=PULSE_AFTER_PULSE 3=PULSE_AFTER_PULSE_AFTER_PULSE 4=TOGGLE
5=PUSH_BUTTON

Nordic UART UUIDs

```
Service 6e400001-b5a3-f393-e0a9-e50e24dcca9a  
RX (write) 6e400002-b5a3-f393-e0a9-e50e24dcca9a  
TX (notify) 6e400003-b5a3-f393-e0a9-e50e24dcca9a
```

מסמך זה נכתב מקריאת קוד המקור. כשמשהו בקוד משתנה (במיוחד [secureProtocol.ts](#), [linkService.ts](#), [gatesStore.ts](#)) — עדכנו גם כאן.*